

RELATÓRIO TÉCNICO DE SEGURANÇA

Relatório de Auditoria de Segurança

atelie-erp

Ateliê ERP + DeskcommCRM — Astro 7 SSR · Cloudflare Workers · Cloudflare D1 · TypeScript

DATA 02 de outubro de 2026	ACHADOS 23	CATEGORIAS 5	ROTAS AUDITADAS 52 arquivos em src/pages/api
-------------------------------	---------------	-----------------	---

Escopo auditado

- Backend.** Os 52 arquivos de `src/pages/api/**` (todas as rotas, sem amostragem), as fábricas `routeFactory.ts` e `crud.ts`, e a camada de autenticação/autorização `middleware.ts`, `auth.ts`, `authz.ts`, `origin.ts`, `rateLimit.ts`.
- Frontend.** Todos os `src/ui/views/**` e `src/ui/**`, os layouts `.astro` e a política de escape (`escapeHtml` / `escapeAtrib` / `escapeAttr`) e a CSP.
- Segredos.** `wrangler.toml`, `.dev.vars`, `.dev.vars.example`, `waha/docker-compose.waha.yml`, `.github/workflows/ci.yml`, `migrations/*.sql`, `scripts/`, documentação e o **histórico git completo** (55 commits).

Nota metodológica — mapeamento das categorias para esta stack

A stack detectada é **Astro 7** em `output: 'server'` sobre **Cloudflare Workers**, com persistência em **Cloudflare D1** (SQLite) acessado por **SQL parametrizado escrito à mão** (`sql.ts/crud.ts`) — **sem ORM e sem query builder**. O frontend é **framework-free**: DOM imperativo + CSS puro, sem React, o que elimina a categoria de sinks `dangerouslySetInnerHTML` e transfere o XSS do frontend para `innerHTML` + Astro `set:html`. A autenticação é **sessão em D1 com token Bearer**, não cookie nem OAuth, e não existe Supabase no app auditado. Cada categoria foi endereçada assim:

Categoria pedida	Equivalente nesta stack
1. Banco sem trança	Não há RLS (o projeto não usa Supabase). O mecanismo de isolamento real foi identificado como sendo <i>a ausência de isolamento</i> : <code>schema single-tenant</code> , <code>listEntities = SELECT *</code> , autorização por papel em <code>authz.ts</code> . Auditamos listagem, busca, agregação, relatório e exportação.
2. Permissão no navegador	Gate de papel existe em <code>authz.ts:requireRole</code> e é cruzado, um a um, com os gates do frontend (<code>isAdmin/canEdit/role</code>) e com o handler de cada endpoint.
3. IDOR	Todos os handlers com <code>:id</code> em path, query ou body foram percorridos (não amostras), rastreando até a tabela do D1 se há filtro de posse.
4. Chaves expostas	Código, config, <code>[vars]</code> do <code>wrangler</code> , <code>.dev.vars</code> , <code>docker-compose</code> , CI, <code>migrations (seed)</code> , <code>scripts</code> , docs e histórico git (<code>git log -S</code>). Defaults inseguros <code>\${VAR:-default}</code> e ausência de validação de startup foram checados.
5. Inputs sem tratamento	Inventário de todos os sinks <code>innerHTML/outerHTML/insertAdjacentHTML/set:html</code> , com rastreamento da <i>origem</i> de cada valor interpolado (API, storage, query string, digitação) e verificação de biblioteca de sanitização.

Relatório gerado por `docs/security-audit/gerar-relatorio.py` — regerável a qualquer momento. Todos os achados foram verificados no código real, com arquivo e linha.

Resumo executivo

Foram registrados **23 achados** em cinco categorias. O perfil é de uma base com **fundamentos sólidos e uma lacuna estrutural**: a autenticação, a criptografia de senha, o rate limit, a CSP, a auditoria e o transporte do WAHA estão bem implementados e verificados. O problema está na **camada de autorização** — e ele se origina em um único lugar: as fábricas que geram as rotas de entidade não recebem papéis, então o único controle real é "ter sessão". Disso decorrem a escalada de privilégio (crítica), a escrita por papel de leitura, e o IDOR em ~24 rotas.



Pontos fortes

O que foi verificado e está **correto**. Esta seção é também a prova de cobertura: cada item é um controle que a auditoria procurou e encontrou implementado com evidência.

CONTROLE	EVIDÊNCIA
Backbone de autenticação no middleware src/middleware.ts:116-136	Todas as rotas `/api/*` exigem sessão válida, exceto 5 caminhos explicitamente listados em PUBLIC_PATHS; o resto responde 401 `nao_autenticado`. O usuário é injetado em `locals` para os handlers.
Rotas públicas se auto-autenticam (cobertura item a item) health.ts:15, webhook-config.ts:26 e :58	Como o middleware não resolve sessão nessas rotas, cada uma valida o token sozinha — e todas as que precisam, validam. A rota de webhook não usa sessão: autentica por HMAC.
Webhook WAHA falha fechado e compara em tempo constante src/server/wahaWebhook.ts:47-64 e :157-162	Assinatura presente e errada é sempre recusada; `requireSignature` recusa payload sem header; `constantTimeEqual` evita vazamento por tempo. O corpo bruto é arquivado antes de qualquer parse.
Anti-SQLi: allowlist de colunas e parâmetros vinculados src/server/mapping.ts:38,48-58 · crud.ts:31,46,65,78	`keepAllowed` descarta qualquer chave fora de `shape.columns`, o que impede contrabandear nome de coluna (e portanto SQL) via JSON. Todo valor vai por `bind()` com placeholder `?`.
Senhas: PBKDF2-SHA256 100k com salt aleatório por usuário src/server/auth.ts:41-46, 218-221	Cada gravação de senha sorteia 16 bytes de sal (`crypto.getRandomValues`) e grava hash+salt; o sal fixo do seed é só fallback de compatibilidade.
Ciclo de vida de sessão previsível src/server/auth.ts:27, 120-147	TTL de 30 dias, `revokeOtherSessions` na troca de senha e `deleteSessionsForUser` no login — uma sessão ativa por usuário.
Rate limit de login em D1, não em memória do isolate src/server/rateLimit.ts · src/middleware.ts:58-64	5 tentativas de login e 3 de troca de senha por janela de 15 min, com contador persistido — resiste ao restart de deploy e à dispersão por colo.
Bloqueio da API enquanto a senha do seed não é trocada src/middleware.ts:25-29, 138-144	`mustChangePassword=1` produz 403 `troca_de_senha_obrigatoria` fora de uma allowlist de três rotas — é o que de fato inutiliza a credencial padrão (C4-01).
Guarda de mesma-origem nas rotas de sessão src/server/origin.ts:10-19	`login`, `logout`, `me` e `change-password` rejeitam `Origin` divergente; clientes sem header (o engine WAHA) passam.
Cabeçalhos de segurança e CSP restritiva src/middleware.ts:31-56	CSP com `script-src 'self'` (sem `unsafe-inline`), `frame-ancestors 'none'`, `base-uri 'self'`, mais HSTS, `nosniff` e `X-Frame-Options`. É o que contém C5-01/C5-02/C5-04 a injeção de script.
Mensagens de WhatsApp — o campo de maior risco — são escapadas CrminboxView.ts:711 e :602, :781 e :794	O texto chega de pessoas externas via webhook WAHA e é renderizado com `escapeHtml(m.text)`; preview da caixa de entrada e corpo das notas também.

Views do CRM escapam antes de interpolar CrmPainelView.ts:91-92, CrmFunilView.ts:101/113/128-130, CrmAgendaView.ts:84/91-92, CrmAtividadesView.ts:94-98, CrmCatalogoView.ts:58-60, CrmContatosView.ts:186-187	O padrão “escapar para uma local, depois interpolar” é consistente nessas telas: nome de contato, título de negócio, etapa, evidência de atividade e descrição de produto saem escapados.
Auditoria das operações privilegiadas src/server/authz.ts:35-45 · routeFactory.ts:22-34	Negação por papel gera entrada `role_denied` com IP; criação de entidade gera action_log com userId e clientId.
O transporte WAHA nunca carrega segredo no erro src/server/waha.ts:20-21, 40-47	`WahaError` só formata `waha_<op>_<status>`; o corpo da resposta — que pode conter telefone, HMAC e API key — é descartado. Timeout em toda chamada.
Placeholder de API key rejeitado na leitura de config src/server/waha.ts:60-70	`readWahaConfig` devolve null quando a chave é `WAHA_DEV_PLACEHOLDER_KEY`, então uma clone fresca não tenta credencial falsa. É o padrão de validação que falta no C4-04.
Compose do WAHA usa sentinela fail-closed, não chave padrão waha/docker-compose.waha.yml:41	`WAHA_API_KEY: "\${WAHA_API_KEY_SHA512:-sha512:INVALID_CHANGE_ME}"` — nenhum hash de chave real casa, então o container recém-criado recusa tudo (401) em vez de abrir com senha padrão.
`dev.vars` nunca entrou no git verificado: `git ls-files --error-unmatch .dev.vars` → erro; `git log --all -S` do prefixo da chave → vazio	A chave WAHA real (C4-02) está na árvore de trabalho mas não no histórico. A CI também não injeta segredo em nenhum bloco `run:` — os jobs de deploy estão comentados (ci.yml:146-209).
LGPD: o escopo do titular é filtrado por atribuição src/pages/api/me/data.ts:51-65 · erase.ts:115-125	Contatos, conversas, negócios, tarefas e consents são recortados por `assignedUserId`/`assigneeUserId`, e o erasure apaga sessões/consents/conta além de anonimizar os registros do titular. O defeito é a exceção, não a regra (C1-01).
Helpers de escape de atributo existem e são usados corretamente em parte do código src/domain/format.ts:39 · src/ui/dom.ts:65 · crmUi.ts:57,76,82	`escapeAtrib` e `escapeAttr` estão presentes e são usados em `crmUi.textField`, dateField, datetimeField, optionHtml e no aria-label do gráfico — o problema é aplicação inconsistente, não ausência (C5-04).

Pontos fracos — os riscos centrais

A partir daqui o relatório sai do campo "o que está bem" e entra no campo "o que precisa mudar". Estes quatro pontos resumem a superfície de risco real do aplicativo — os demais achados são consequência deles.

Autorização é a camada fraca

Quatro arquivos no app inteiro chamam `requireRole`. Todas as demais rotas — que são a maioria — confiam apenas no middleware. A hierarquia de papéis foi declarada (`ROLE_RANK`) e nunca usada, e a matriz de permissões está duplicada em literais na UI e no servidor. É exatamente esse arranjo que deixou a escalada de privilégio passar.

"Single-tenant" é uma premissa silenciosa

O app é single-tenant por desenho, e isso está documentado. O risco é que a premissa se propaga sozinha: toda rota nova herdada das fábricas nasce global, e `assignedUserId/assigneeUserId` parecem fronteira de segurança quando são só metadado de atribuição. A rota de privacidade LGPD é a prova de que a distinção importa — e ela falhou (C1-01).

Escape de HTML é aplicado por convenção, não por garantia

Não há biblioteca de sanitização no projeto. O que existe são três helpers de escape, dois deles divergentes, e a aplicação é manual: a maioria das views escapa corretamente, mas `openModal`, o Dashboard, a `LoginView` e os campos numéricos foram esquecidos. Uma regra de lint fecharia essa classe inteira.

Sessão em `localStorage` é o multiplicador

Enquanto o token estiver em `localStorage`, qualquer injeção de HTML que evolua para execução de script vira comprometimento de conta. A CSP atual (`script-src 'self'`) é o que separa "deformar HTML" de "roubar sessão" — um controle único segurando o impacto de quatro achados.

Achados detalhados por categoria

Cada linha cita o arquivo e a linha exatos, o trecho verificado e a condição de explorabilidade. Os achados marcados como *verificação positiva* existem para registrar a cobertura da auditoria.

1. Banco sem trança (isolamento de inquilino/dono)

ID	SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
C1-01	Alta	src/pages/api/me/data.ts:73 src/pages/api/me/data.ts:76-77 src/pages/api/me/export.ts:68 src/pages/api/me/export.ts:71-72	Exportação LGPD devolve mensagens, clientes e pedidos de TODO o sistema O recorte LGPD filtra contatos, conversas, negócios, tarefas e consents por <code>`assignedUserId`/`assigneeUserId` (selectUserRows)</code>, mas os campos <code>`messages`</code>, <code>`customers`</code> e <code>`orders`</code> são devolvidos SEM nenhum filtro — vêm de <code>`all.*`</code>, que é a base inteira carregada por <code>`loadLgpdData`</code>. O módulo existe justamente para devolver apenas os dados do titular, então a rota entrega mais do que promete: qualquer sessão autenticada, de qualquer papel, recebe a lista completa de clientes (nome/telefone/e-mail) e de pedidos do D1. É um vazamento de dados pessoais entre usuários, exposto por uma rota de privacidade. Explorabilidade: Qualquer usuário autenticado (inclusive <code>`viewer`</code>) chama <code>GET /api/me/export</code> ou <code>GET /api/me/data</code>. Não exige feature flag. Correção: Filtrar <code>`messages`</code> pelo conjunto de conversas do titular e derivar <code>`customers`/`orders`</code> dos contatos filtrados, ou remover esses três campos do recorte. Adicionar teste que falhe se um usuário sem atribuição receber qualquer linha de <code>customers/orders/messages</code>. ► trecho de código
C1-02	Média	src/pages/api/crm/events.ts:31-38 src/pages/api/crm/events.ts:172 src/pages/api/crm/events.ts:137	SSE faz broadcast de todas as mensagens e do contato (telefone/nome) para qualquer sessão O stream de eventos não tem recorte: ele emite <code>`m.*`</code> (todas as mensagens do sistema) junto com <code>`contact_name`/`contact_phone`</code> de todos os contatos, para qualquer sessão autenticada. Some-se a isso <code>`Access-Control-Allow-Origin: *`</code> e ao token transportado por query string (<code>`?token=`</code>), que é o único jeito de o EventSource autenticar. O token em URL vaza para log de acesso, header Referer e histórico do navegador — exatamente o risco que o comentário do próprio middleware reconhece ao justificar a exceção. Explorabilidade: Sessão autenticada + EventSource aberto. <code>ACA:*</code> significa que qualquer origem que consiga o token lê o stream. Correção: Trocar a exceção de token-em-URL por um cookie <code>`HttpOnly; Secure; SameSite=Strict`</code> de sessão (o EventSource envia cookies), removendo o <code>`?token=`</code> e o <code>ACA:*</code>. ► trecho de código

C1-03	Baixa	src/server/crud.ts:16 migrations/0003_crm.sql:2	Mecanismo de isolamento é a ausência de isolamento: listEntities devolve a tabela inteira Não existe RLS (o projeto não usa Supabase), nem middleware de tenant, nem coluna owner/org — o schema declara single-tenant e todas as listagens são um <code>`SELECT *`</code> sem filtro. Isto é uma DECISÃO DE PROJETO documentada no README ("As listagens devolvem a base inteira"), não um descuido. O risco é que essa premissa é silenciosa: qualquer rota nova herdada de <code>createCollectionRoutes</code> nasce global, e <code>`assignedUserId`/`assigneeUserId`</code> parecem fronteira de segurança mas são só metadado de atribuição. Explorabilidade: Qualquer sessão válida vê a base inteira. Aceito por desenho; registrado para que não seja tratado como garantia. Correção: Manter, mas registrar a decisão como invariante: documente que qualquer rota nova precisa de um gate explícito de papel, e considere um teste que falhe se um <code>`viewer`</code> conseguir escrever. ► trecho de código
-------	-------	--	--

2. Permissão definida no navegador

ID	SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
C2-01	Crítica	src/pages/api/users/index.ts:13 src/pages/api/users/index.ts:31 src/pages/api/users/index.ts:12 src/pages/api/users/index.ts:39-41 src/pages/api/users/[id].ts:62-65	Escalada de privilégio: `manager` cria conta `admin` em POST /api/users (o PUT equivalente exige admin) A rota de criação aceita <code>`['admin','manager']`</code> e valida o papel pedido apenas contra a lista de papéis válidos — que inclui <code>`admin`</code>. Um <code>`manager`</code> (papel intermediário, não-administrador) envia POST com <code>`role:'admin'`</code> e uma senha escolhida por ele, criando um administrador full. O próprio projeto já corrigiu isso na rota irmã: <code>`PUT /api/users/[id]`</code> usa <code>`allowedRoles()`</code>, que exige <code>`['admin']`</code> quando o corpo traz <code>`role`</code> ou <code>`password`</code>. O POST ficou de fora dessa regra — a validação existe no código e simplesmente não foi aplicada aqui. A UI não é a fronteira: <code>`CrmEquipeView`</code> oferece os quatro papéis no select (<code>roleOptions()</code>, linhas 237-242) e não limita a escolha por papel do operador. Explorabilidade: Qualquer sessão com papel <code>`manager`</code>. Sem feature flag: o gate real é só a lista <code>USER_MANAGERS</code>. Correção: Reusar <code>`allowedRoles(body)`</code> no POST (ou exigir <code>`ADMINS`</code> quando <code>`body.role`</code> for <code>`admin`</code>), e adicionar teste que falhe para POST com <code>role=admin</code> por <code>manager</code>. ► trecho de código

C2-02 **Alta**

```
src/server/routeFactory.ts:39-57
src/server/routeFactory.ts:59-80
src/ui/views/crm/CrmEquipeView.ts:12
src/domain/crm.ts:5
```

Papel `viewer` (“Visualização”) tem poder de escrita e exclusão em todas as entidades

As fábricas que geram TODAS as rotas de entidade (contatos, conversas, mensagens, negócios, tarefas, pipelines, etapas, etiquetas, agenda, catálogo, atividades, notas, clientes, pedidos, produtos, ingredientes, componentes, movimentos de estoque) não chamam `requireRole`. Verificado por busca: `requireRole` aparece apenas em `users/index.ts`, `users/[id].ts`, `settings.ts` e `whatsapp/session.ts`. Logo o único controle é o middleware, que exige uma sessão válida — e um papel chamado “Visualização” pode `DELETE` `/api/crm/contacts/<id>`, `POST` `/api/crm/pipelines` e apagar pedidos. O rótulo comercial diz o que o papel não cumpre.`

Explorabilidade: Qualquer sessão válida, de qualquer papel. Não há feature flag: a ausência de gate é total nas rotas geradas.

Correção: Dar a `createCollectionRoutes/createItemRoutes` uma lista de papéis permitidos por operação (leitura: todos; escrita: `agent+`; exclusão: `manager+`) e validar com `requireRole` antes de tocar o D1.

► [trecho de código](#)

C2-03 **Média**

```
src/ui/views/crm/CrmEquipeView.ts:102-104
src/pages/api/users/[id].ts:50
src/pages/api/users/[id].ts:17
```

Gate de papel do frontend mais permissivo que o endpoint em excluir usuário

A tela de Equipe mostra o botão Excluir para `manager`, mas o endpoint exige `[admin]`. O servidor está certo e a UI mente sobre a capacidade — o operador clica e leva 403. É o espelho do C2-01: aqui o frontend é mais permissivo, lá o frontend é permissivo e o backend também. Nos dois casos a verdade está no servidor, mas a inconsistência mostra que o gate de papel não é uma fonte única.

Explorabilidade: Qualquer sessão `manager` na tela `/equipe`.

Correção: Derivar a lista de papéis de uma constante compartilhada (ex.: um `can(user, action)` no servidor consumido pela UI), em vez de repetir literais nos dois lados.

► [trecho de código](#)

C2-04	Média	<pre>src/ui/views/crm/CrmEquipeView.ts:42-47 src/ui/views/crm/CrmEquipeView.ts:139</pre>	Botão “+ Novo usuário” é renderizado sem nenhum gate de papel `pageHead()` não recebe o contexto nem o usuário, e monta o botão sempre. Um `viewer` vê e clica em “+ Novo usuário”; só o 403 do servidor impede a criação. Nenhuma UI de papel está escondendo a ação privilegiada — a autorização é inteiramente tardia e invisível para o operador. Explorabilidade: Qualquer sessão autenticada na rota /equipe. Correção: Condicionar o botão ao papel do usuário atual e desabilitar o submit quando o POST voltar 403. ► trecho de código
C2-05	Baixa	<pre>src/domain/crm.ts:166-171</pre>	ROLE_RANK` existe mas nunca é usada — a hierarquia de papéis não é aplicada</b></p> <p>Busca em `src/` e `tests/` retorna apenas a definição: nenhum consumidor. O rank foi feito para decisões de autorização do tipo “manager <math>\supseteq</math> agent”, mas os gates comparam listas explícitas (`['admin','manager']`), então a hierarquia é código morto. Isso enfraquece o argumento de que “o papel é a fronteira de segurança”: o modelo de papéis é informal e só vale o que cada lista escrita à mão disser.</p> <p><b>Explorabilidade:</b> N/A — código morto.</p> <p><b>Correção:</b> Usar `ROLE_RANK[user.role] >= ROLE_RANK[required]` em `requireRole`, ou remover ROLE_RANK e a menção a hierarquia. ► trecho de código

3. IDOR

ID	SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
C3-01	Alta	src/server/routeFactory.ts:60-72 src/server/routeFactory.ts:74-77 src/pages/api/crm/contacts/[id].ts:3 src/pages/api/orders/[id].ts:3	createItemRoutes faz PUT/DELETE por id sem checar posse nem papel (24 rotas [id]) O `id` vem de `context.params` e vai direto para o UPDATE/DELETE, sem nenhuma comparação de `assignedUserId` / `assigneeUserId` nem gate de papel. Todos os handlers `[id].ts` são um alias de duas linhas para essa fábrica — foram percorridos todos: contatos, conversas, mensagens, negócios, tarefas, pipelines, etapas, respostas-rápidas, agenda, catálogo, atividades, notas, etiquetas, tipos de agenda, clientes, pedidos, produtos, ingredientes, componentes, usuários. Efeito concreto: um `viewer` apaga qualquer contato, qualquer pedido ou qualquer negócio pelo id. Também não há checagem de propriedade para registros que TEM dono declarado (`contacts.assignedUserId`, `tasks.assigneeUserId`, `deals.assignedUserId`). Explorabilidade: Qualquer sessão válida + conhecimento do id (os ids são devolvidos pelas listagens, que não filtram). Correção: Antes do write, resolver o registro e exigir (a) papel mínimo para a operação e (b) quando a tabela tiver coluna de dono, `registro. <dono> === user.id    user.role em ('manager','admin')`. Centralizar na fábrica para não depender de cada rota. ► trecho de código
C3-02	Alta	src/pages/api/whatsapp/send.ts:26-31 src/server/wahaSend.ts:62-79 src/pages/api/whatsapp/send.ts:15-16	IDOR no envio de WhatsApp: qualquer usuário envia mensagem em qualquer conversa `conversationId` vem do corpo e é resolvido por id. `loadWahaConversation` valida apenas existência e `channel === 'whatsapp'` — nunca compara `conversation.assignedUserId` com o chamador (confirmado: a string `assignedUserId` não aparece no arquivo). A rota também não tem `requireRole` (só sessão). Combinado com o C1-02/C1-03 (listagens devolvem a base inteira), qualquer `viewer` pode listar as conversas e então disparar WhatsApp para qualquer cliente do negócio — um canal comercial externo, com custo e repercussão. Explorabilidade: Sessão autenticada de qualquer papel. Sem feature flag. Correção: Exigir `assignedUserId === user.id    role em ('manager','admin')` em `loadWahaConversation` e aplicar `requireRole(['agent','manager','admin'])` na rota. ► trecho de código

C3-03

Média

src/pages/api/settings.ts:13-19
src/pages/api/settings.ts:25-28
src/server/mapping.ts:9-10

GET /api/settings escreve no banco e faz merge cego do patch

Duas coisas numa rota só de leitura: o GET grava (INSERT OR REPLACE) quando a configuração ainda não existe — estado que muda em resposta a um GET, sem gate de papel nenhum (o GET não tem requireRole). E o PUT faz merge de qualquer chave do corpo sobre o registro, porque `settings` é a única tabela sem `columns` no shape: `keepAllowed` retorna `{ ...entity }` quando não há allowlist. O PUT tem gate de manager/admin, mas o objeto gravado aceita colunas arbitrárias — o atacante escolhe o que sobrescrever dentro da linha.

Explorabilidade: Leitura por qualquer sessão; escrita por `manager`/`admin`.

Correção: Mover a criação do registro default para uma migration/seed; dar a `settings` uma lista explícita de colunas; validar o patch contra um schema.

► [trecho de código](#)

C3-04

Baixa

src/middleware.ts:15-21
src/pages/api/whatsapp/health.ts:15-16
src/pages/api/whatsapp/webhook-config.ts:26-27

Cobertura de auth verificada item a item — 5 rotas públicas se auto-autenticam

Rotas fora de PUBLIC_PATHS recebem `locals.user` do middleware e não precisam se autenticar. As 5 de PUBLIC_PATHS pulam essa resolução, então cada uma tem que validar o token por conta própria — e todas as quatro que precisam, fazem. `webhook` (POST) não usa sessão: é autenticado por HMAC. Este achado registra a verificação positivamente, e o ponto de atenção é o inverso: como o middleware não roda a checagem de `mustChangePassword` nessas rotas, elas dependem da própria rota — uma rota nova adicionada a PUBLIC_PATHS sem `userFromToken` ficaria aberta sem erro de compilação.

Explorabilidade: N/A — verificação positiva.

Correção: Converter a lista em negativa: em vez de PUBLIC_PATHS, exigir sessão por padrão (já é o caso) e teste que falhe se uma rota pública não chamar userFromToken.

► [trecho de código](#)

4. Chaves expostas (hardcode)

ID	SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
C4-01	Alta	migrations/0004_crm_seed.sql:5-9 src/server/auth.ts:26 src/server/auth.ts:48-52	Credencial de administrador padrão documentada no repositório, com salt fixo publicado no código O seed grava um admin com senha `admin123` e sal FIXO `deskcomm-seed-v1` — e esse mesmo sal está no código-fonte, num default de função. `verifyPassword` cai de volta nesse sal padrão quando a linha não tem `passwordSalt`, então o credencial é reproduzível por qualquer pessoa com o repositório. A mitigação existe e é boa: `mustChangePassword=1` (0020/0021) + middleware respondendo 403 `troca_de_senha_obrigatoria` fora da allowlist. O que não existe é validação de startup que rejeite o credencial de seed: o login retorna 200 e um token de sessão válido — só a API fica fechada. Explorabilidade: Explorável quando `npm run db:seed:remote` roda em ambiente novo sem o operador trocar a senha. O comentário de 0019 deixa o hash exposto de propósito para o UPDATE idempotente. Correção: Não semear senha: gerar o admin no primeiro boot com senha aleatória exibida uma vez, ou exigir `ADMIN_INITIAL_PASSWORD` no env. Adicionar asserção de startup que aborta se `users.passwordHash` do seed ainda estiver lá. ► trecho de código
C4-02	Média	.dev.vars:9 .gitignore:5	Chave WAHA real (64 hex) presente no arquivo de trabalho `.dev.vars` O arquivo contém uma credencial de formato real, não o placeholder `dev_plaintext_change_me` do `.dev.vars.example`. Verificado: `.dev.vars` NÃO é rastreado por git (`git ls-files --error-unmatch` → erro) e NÃO aparece em nenhum commit (`git log --all -S'7ff62014c4d63e71'` vazio). O controle de versionamento está correto; o risco é operacional — a chave de desenvolvimento fica em texto plano na árvore de trabalho e em qualquer backup/cópia da pasta. Explorabilidade: Qualquer pessoa/leitor com acesso ao disco ou a um backup do diretório. Não é exposto por git. Correção: Mover a chave de desenvolvimento para `wrangler secret put`/gerenciador de senhas e rodar o WAHA de dev com um `.env` fora do repositório. ► trecho de código

C4-03	Média	DeskcommCRM-RecipeCosting/.env.example:1 DeskcommCRM-RecipeCosting/.env.hostgator.example:1 DeskcommCRM-RecipeCosting/docker-compose.prod.yml:1	App Next.js/Supabase inteiro vendorizado dentro do mesmo repositório git (4807 arquivos) O repositório do atelie-erp carrega uma cópia completa de um segundo produto (DeskcommCRM: Next.js + Supabase + Sentry), com 4807 arquivos rastreados (verificado): <code>`.env.example`</code>, <code>`.env.hostgator.example`</code>, <code>`.supabase/migrations/`</code>, <code>`.Dockerfile*`</code>, <code>`.docker-compose.prod.yml`</code> e o histórico de CHANGELOG/HANDOFF. Os JWTs encontrados são fixtures públicos do Supabase (chave de dev documentada) e placeholders (<code>`chave-de-mentira`</code>), não credenciais reais — mas isso é sorte, não controle: o diretório é superfície de segredo e de ataque sem nenhuma revisão, e qualquer scanner de segredo/DAST passa a produzir ruído de um produto que nem está no escopo deste deploy. Explorabilidade: Qualquer pessoa com acesso ao repositório recebe também o histórico desse segundo app. Correção: Remover <code>`.DeskcommCRM-RecipeCosting/`</code> do repositório do atelie-erp (git <code>rm -r --cached</code> e adicionar ao <code>.gitignore</code>), ou movê-lo para um repositório próprio e consumi-lo como submódulo/subtree explícito. ▶ trecho de código
C4-04	Baixa	<code>.dev.vars:31</code> <code>src/server/wahaWebhook.ts:29-35</code>	Placeholder de <code>`WAHA_HMAC_SECRET`</code> é aceito como segredo válido <code>`text()`</code> só checa string não-vazia, então o texto de instrução <code>`gere-um-segredo-...`</code> passa a valer como segredo. Neste ambiente específico o efeito é fail-closed (bom): com <code>`WAHA_WEBHOOK_REQUIRE_SIGNATURE="true"`</code> e segredo presente, todo payload sem assinatura válida é recusado com 401. O padrão é que frágil: "não vazio" é o único critério de validade, e se alguém trocar o placeholder por um segredo fraco (ou colar a mesma string em vários ambientes) nada reclama — e o <code>`.dev.vars.example`</code> deixa o campo vazio, o que DESLIGA a verificação. Explorabilidade: Requer um valor não-vazio e errado. Com o valor atual o efeito é negação de serviço, não entrada forjada. Correção: Rejeitar valores conhecidos de placeholder e segredos com menos de 32 bytes, como <code>`readWahaConfig`</code> já faz para a API key (<code>`WAHA_DEV_PLACEHOLDER_KEY`</code>). ▶ trecho de código

5. Inputs sem tratamento (XSS)

ID	SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
C5-01	Alta	src/ui/Modal.ts:28-31 src/ui/views/crm/CrmContatosView.ts:302 src/ui/views/StockView.ts:169-170	openModal interpola <code>`title`</code> sem escape; dois callers passam nome vindo do banco <code>`options.title`</code> cai direto no innerHTML do backdrop. Os dois callers que passam dado do banco — o nome do contato e o nome do ingrediente — não escapam nada. Como qualquer sessão autenticada (mesmo <code>`viewer`</code>) escreve qualquer <code>`name`</code> via <code>`POST/PUT /api/ingredients/:id`</code> e <code>`POST/PUT /api/crm/contacts/:id`</code> (nenhum dos dois exige papel), o nome atacante fica armazenado e é renderizado em HTML para todo usuário que abrir aquele formulário — injeção de HTML armazenada. <code>`CrmEquipeView`</code> mostra o contraste: a linha 65-66 escapa <code>`me.name`/`me.email`</code> antes de usar. Explorabilidade: Requer CSP relaxada para virar XSS de <code>script: `script-src 'self'`</code> (middleware.ts:31-34) bloqueia handlers inline. A injeção de HTML/atributo funciona sempre. Correção: Escapar <code>`options.title`</code> dentro de Modal.ts (<code>`<h3>\${escapeHtml(options.title)}</h3>`</code>), e escapar <code>`ingredient.name`/`contact.name`</code> nos callers. ► trecho de código
C5-02	Alta	src/ui/views/DashboardView.ts:86-88	DashboardView renderiza <code>`i.name`</code> (ingrediente) cru no innerHTML O painel do Ateliê lista os ingredientes em estoque baixo interpolando o nome sem escape algum — nem local, nem direto. <code>`i.name`</code> e <code>`i.unit`</code> são graváveis por qualquer sessão autenticada via <code>`/api/ingredients/:id`</code>, sem gate de papel. O vetor é o mesmo do C5-01 e o impacto é maior, porque o Dashboard é a primeira tela que qualquer usuário vê ao entrar. Explorabilidade: Qualquer sessão válida planta o payload; a vítima precisa abrir o Dashboard do Ateliê. Correção: <code>`escapeHtml(i.name)`</code> e <code>`escapeHtml(i.unit)`</code> — o mesmo padrão já usado em <code>`CustomersView`</code> e <code>`CrmInboxView`</code>. ► trecho de código

C5-03

Média

```
src/ui/views/LoginView.ts:44-45
src/pages/api/users/[id].ts:18
src/pages/api/users/[id].ts:88-94
```

LoginView renderiza `me.name` e `me.email` crus (injeção cross-user via PUT /api/users)

O `me` é o próprio usuário logado, o que sugeriria self-XSS inofensivo. Não é: `editablePatch` permite que `manager`/`admin` reescreva o `name` e o `email` de **qualquer** usuário via `PUT /api/users/[id]` (só `role`/`password` exigem admin). Um manager planta o payload no registro de um colega; quando esse colega entra e a LoginView renderiza o perfil, o HTML é injetado. A CrmEquipeView (65-66) escapa o mesmo dado — a LoginView é a cópia esquecida.

Explorabilidade: Requer um `manager`/`admin`-planted payload e a vítima entrando por LoginView.

Correção: `escapeHtml(me.name)` / `escapeHtml(me.email)`, ou deletar LoginView se CrmEquipeView é a tela de sessão canônica.

[▶ trecho de código](#)

C5-04

Média

```
src/domain/format.ts:33-37
src/ui/views/crm/CrmEtiquetasView.ts:91
src/ui/views/ProductsView.ts:508-510
src/ui/views/crm/CrmWhatsAppView.ts:308-310
```

`escapeHtml` não escapa aspas e é usado dentro de atributos → quebra de atributo

`escapeHtml` escapa via `textContent→innerHTML`, que trata `&`, `<` e `>` mas **NÃO** `"` nem `` — porque aspas só importam em contexto de atributo. Por isso o projeto tem `escapeAtrib` (format.ts:39) e `escapeAttr` (dom.ts:65). Nos três pontos acima o valor vai para dentro de aspas de atributo usando o helper errado: `tag.color` num `class=` (gravável por qualquer autenticado em `PUT /api/crm/tags/:id`), `value=` de input de produto, e o `src=` do QR do WhatsApp (controlado pelo engine WAHA, alcançado por HTTP simples em dev). Um `` no payload fecha o atributo e abre Markup.

Explorabilidade: tag.color e os valores de produto são graváveis por qualquer sessão; o QR exige controlar a resposta do engine (ou MITM em dev, onde a base URL é `http://localhost:3000`).

Correção: Trocar por `escapeAtrib`/`escapeAttr` nos três pontos e unificar os dois helpers de atributo em um só.

[▶ trecho de código](#)

C5-05

Baixa

```
src/ui/views/crm/crmUi.ts:69
src/ui/views/ProductsView.ts:521
src/ui/views/IngredientsView.ts:195
src/ui/views/ComponentsView.ts:316
src/ui/views/SettingsView.ts:97
```

numberField interpola `value` sem escape em cinco views

O campo `value` de todo `<img src=x onerror=...>"}` grava a string e ela volta no formulário. `textField`, no mesmo arquivo, usa `escapeHtml` e `crmUi.textField` usa `escapeAtrib` — a lacuna é só no `numberField`.

Explorabilidade: Escrita por qualquer sessão autenticada (sem gate de papel nas rotas de entidade).

Correção:

`value`="\${escapeAtrib(String(value))}"` nas cinco implementações, e validação de tipo no servidor para os campos numéricos.

► [trecho de código](#)

C5-06

Baixa

```
src/ui/views/crm/crmUi.ts:128-131
```

rowButton interpola o valor do atributo `data-\*` sem escape

O helper compartilhado de botão de linha coloca `value` dentro de aspas de atributo sem escapar. Os valores atuais são ids de entidade gerados por `uid()`, então não há caminho praticável hoje — registrado porque o helper é usado por várias views e o contrato não documenta essa exigência.

Explorabilidade: Não explorável com os ids atuais; passaria a ser se algum caller passar texto.

Correção:

`data-\${dataset}="\${escapeAtrib(value)}"` e um comentário exigindo id/texto já escapado.

► [trecho de código](#)

C5-07

Média

src/repositories/AuthRepository.ts:20
src/repositories/AuthRepository.ts:56

Token de sessão em localStorage — qualquer injeção de HTML na origem vira sequestro de sessão

O token Bearer vive em `localStorage`, legível por qualquer script que rode na origem. Os sinks de innerHTML verificados injetam HTML mas a CSP atual (`script-src 'self'`) barra a execução de script, então hoje o pior caso é injeção de HTML/form. O ponto é que o nível de dano é alto: se a CSP relaxar, ou se um sink passar a permitir `javascript:`, o mesmo bug vira account takeover completo em vez de simples deformação de HTML. `crypto.randomUUID()` para o token não muda isso.

Explorabilidade: N/A — é um amplificador dos achados C5-01..C5-04.

Correção: Migrar a sessão para cookie `HttpOnly; Secure; SameSite=Strict` (o que também resolve o token-em-URL do SSE, C1-02).

► [trecho de código](#)

Recomendações priorizadas

P1 fecha o vetor de comprometimento total (escalada de privilégio e vazamento de dados). **P2** fecha IDOR e injeção de HTML. **P3** é higiene e defesa em profundidade — os itens que reduzem a superfície e impedem a reintrodução das mesmas classes de falha.

PRIOR.	ACHADOS	AÇÃO	POR QUÊ
P1	C2-01	Fechar a escalada de privilégio em POST /api/users: exigir papel `admin` sempre que `body.role === 'admin'`, reaproveitando `allowedRoles()` do PUT irmão. Adicionar teste que falhe para manager→admin.	Evita controle total do app por um papel intermediário.
P1	C1-01	Corrigir o recorte de /api/me/data e /api/me/export: derivar `messages` das conversas do titular e `customers`/`orders` dos contatos filtrados (ou removê-los). Teste que falhe para usuário sem atribuição.	Fecha o vazamento de PII e a exposição de todos os pedidos.
P1	C2-02	Dar papéis permitidos às fábricas `createCollectionRoutes`/`createItemRoutes` e validar com `requireRole` por operação (leitura todos; escrita agent+; exclusão manager+).	Faz o papel `viewer` significar “visualização” e fecha a escrita por papel baixo.
P1	C3-01	Na fábrica de item routes, resolver o registro e exigir posse (`<dono> === user.id` ou manager/admin) quando a tabela tiver coluna de dono, além do gate de papel.	Fecha IDOR em ~24 rotas de uma vez, sem depender de cada arquivo.
P2	C3-02	Exigir posse em `loadWahaConversation` e `requireRole(['agent','manager','admin'])` em POST /api/whatsapp/send.	Impede disparo de WhatsApp para qualquer cliente por qualquer sessão.
P2	C5-01 C5-02	Escapar `options.title` dentro de Modal.ts e `i.name`/`i.unit` em DashboardView. Adicionar lint forbidding interpolação sem escape em innerHTML.	Elimina as duas injeções de HTML armazenado de maior alcance.
P2	C5-04 C5-05 C5-06	Trocar `escapeHtml` por `escapeAtrib`/`escapeAttr` nos contextos de atributo, unificar os dois helpers de atributo e escapar `value` nos `numberField`.	Fecha a quebra de atributo e o vetor de ` <input type="number"/> `.
P2	C4-01	Eliminar a senha de seed: gerar o admin no primeiro boot ou exigir `ADMIN_INITIAL_PASSWORD`; adicionar asserção de startup que rejeite o hash de seed.	Tira um credencial admin reproduzível do repositório.
P2	C2-03 C2-04	Unificar o gate de papel numa função compartilhada `can(user, action)` consumida pela UI e pelo servidor, e condicionar o botão “+ Novo usuário”.	Elimina a divergência UI↔servidor que produziu o C2-01.
P3	C1-02 C5-07	Migrar a sessão para cookie `HttpOnly; Secure; SameSite=Strict`, o que elimina o `?token=` do SSE e o `Access-Control-Allow-Origin: *`.	Reduz drasticamente o impacto de qualquer futura injeção de script.

P3	C3-03	Mover o default de settings para migration, dar allowlist de colunas a `settings` e validar o patch contra um schema.	Retira escrita em rota de leitura e a gravação de colunas arbitrárias.
P3	C4-03	Remover `DeskcommCRM-RecipeCosting/` do repositório (ou movê-lo para repo próprio), eliminando 4807 arquivos de superfície e de ruído de secrets.	Reduz exposição de segredo e ruído de scanner/DAST.
P3	C4-04 C2-05	Rejeitar placeholders e segredos curtos em `readWahaWebhookConfig`; usar `ROLE_RANK` em `requireRole` ou remover a hierarquia morta.	Fecha defaults frouxos e dá semântica real aos papéis.
P3	C3-04	Teste que falhe se uma rota adicionada a PUBLIC_PATHS não chamar `userFromToken`, garantindo a invariante de cobertura.	Protege contra regressão na lista pública.

Issues para o GitHub

Texto completo de cada issue, pronto para copiar e colar. Cada bloco delimitado por `--- ISSUE n ---` e `--- FIM ISSUE n ---`. Achados triviais relacionados foram agrupados em uma issue única (issues 7, 9 e 12) para não gerar spam.

ISSUE 1

```
--- ISSUE 1 ---
# [Segurança] Escalada de privilégio: `manager` cria conta `admin` em POST /api/users

**Labels sugeridas:** `security` · `critical` · `authz` · `privilege-escalation`

## Problema

A rota de criação de usuário aceita os papéis `[ 'admin', 'manager' ]` e valida o papel pedido apenas contra a lista de papéis **válidos**, que inclui `admin`. Não há nenhuma verificação de que o papel pedido está dentro do que o chamador pode conceder. Um usuário com papel `manager` – que não é administrador – cria uma conta `admin` com senha escolhida por ele e passa a controlar o aplicativo inteiro.

A rota irmã `PUT /api/users/[id]` já resolve isso corretamente: `allowedRoles()` exige `[ 'admin' ]` quando o corpo traz `role` ou `password`. A POST ficou de fora dessa regra – a validação existe no próprio código e não foi aplicada ali. A UI não é fronteira: `CrmEquipeView.roleOptions()` oferece os quatro papéis no select sem limitar por papel do operador, e o botão “+ Novo usuário” é renderizado sem gate algum.

## Evidência

```ts
src/pages/api/users/index.ts:13
 const USER_MANAGERS: Role[] = ['admin', 'manager'];

src/pages/api/users/index.ts:31
 const denied = await requireRole(context, USER_MANAGERS);

src/pages/api/users/index.ts:12
 const ROLES: Role[] = ['viewer', 'agent', 'manager', 'admin'];

src/pages/api/users/index.ts:39-41
 if (!ROLES.includes(body.role as Role)) {
 return json({ error: 'papel_invalido' }, 400);
 }

Contraste – a rota PUT restringe (src/pages/api/users/[id].ts:62-65):
 function allowedRoles(body: PutBody): Role[] {
 const adminOnly = ADMIN_ONLY_FIELDS.some(f => body[f] !== undefined);
 return adminOnly ? ADMINS : USER_MANAGERS;
 }
...

Impacto

Controle total do aplicativo: um `manager` comprometido ou mal-intencionado vira administrador, o que dá acesso a gestão de usuários, configurações e à sessão do WhatsApp (`/api/whatsapp/session` é admin-only, e parar/iniciar a engine derrubaria o canal comercial). Também permite preparar uma conta persistente que sobrevive à troca de senha da vítima – o reset de senha do dono não derruba a sessão de um admin novo.

Sugestão de correção

Aplicar `allowedRoles(body)` no POST (ou exigir `ADMINS` quando `body.role === 'admin'`), e derivar o limite de um único lugar.

Critérios de aceite

- [] POST /api/users com role='admin' por sessão `manager` retorna 403 `papel_insuficiente`.
- [] POST /api/users com role='manager'|'agent'|'viewer' por sessão `manager` continua 201.
- [] Teste automatizado no mesmo estilo de tests/server/authz.test.ts, com uma sessão `manager` no contexto.
- [] O mesmo corpo enviado via PUT com role='admin' por `manager` também retorna 403 (paridade das duas rotas).
- [] A UI esconde a opção `admin` no select quando o operador não é admin.
--- FIM ISSUE 1 ---
```

### ISSUE 2

```
--- ISSUE 2 ---
[Segurança] Exportação LGPD (`/api/me/data`, `/api/me/export`) devolve mensagens, clientes e pedidos de todo o sistema
```

**\*\*Labels sugeridas:\*\*** `security` · `high` · `lgpd` · `data-leak` · `broken-access-control`

## ## Problema

As duas rotas de privacidade do titular aplicam um recorte por atribuição (`assignedUserId` / `assigneeUserId`) a contatos, conversas, negócios, tarefas e consents – mas devolvem `messages`, `customers` e `orders` **\*\*sem nenhum filtro\*\***, direto do conjunto `all.\*`, que é a base inteira carregada por `loadLgpdData`.

Na prática, qualquer sessão autenticada – inclusive um usuário com papel `viewer`, que por definição não deveria ver nada – consegue baixar a lista completa de clientes (nome, telefone, e-mail) e de pedidos do D1 chamando `GET /api/me/export`. Isso é pior que não filtrar: a rota é justamente o endpoint que um titular usaria para exercitar seu direito de acesso, e ele vazava a base de terceiros.

## ## Evidência

```ts

```
src/pages/api/me/data.ts:73
  messages: all.messages,
```

```
src/pages/api/me/data.ts:76-77
  customers: all.customers,
  orders: all.orders,
```

```
src/pages/api/me/export.ts:68
  messages: all.messages,
```

```
src/pages/api/me/export.ts:71-72
  customers: all.customers,
  orders: all.orders,
```

0 recorte que existe, ao lado (src/pages/api/me/data.ts:51-65), mostra o padrão correto:

```
const myContacts = all.contacts.filter(c => c.assignedUserId === user.id);
deals: all.deals.filter(d => d.assignedUserId === user.id),
tasks: all.tasks.filter(t => t.assigneeUserId === user.id),
...
```

Impacto

Vazamento de dados pessoais entre usuários e exposição de informação comercial (volume e valores de pedidos). Sob LGPD (art. 18 e art. 6º, VII) isso é tratamento de dado de terceiro sem base legal e um incidente notificável; um `viewer` já acessa tudo que a rota devolve.

Sugestão de correção

Filtrar `messages` pelo conjunto de conversas do titular e derivar `customers`/`orders` dos contatos filtrados – ou remover esses campos do recorte, já que o titular não tem relação com pedidos de outros.

Critérios de aceite

- [] Um usuário sem nenhuma atribuição recebe `customers`, `orders` e `messages` vazios em `/api/me/data` e `/api/me/export`.
 - [] Um usuário com contatos atribuídos recebe apenas as mensagens das conversas desses contatos.
 - [] Nenhum campo de retorno contém linhas cujo `assignedUserId`/dono é de outro usuário.
 - [] Teste automatizado que falhe se uma linha de customers/orders/messages de terceiro aparecer no payload.
- FIM ISSUE 2 ---

ISSUE 3

--- ISSUE 3 ---

[Segurança] Papel `viewer` tem escrita e exclusão em todas as entidades (routeFactory sem requireRole)

****Labels sugeridas:**** `security` · `high` · `authz` · `missing-authorization`

Problema

Todas as rotas de entidade do CRM e do ERP são geradas por `createCollectionRoutes` e `createItemRoutes`, e ****nenhuma das duas chama `requireRole`****. O único controle sobre elas é o middleware, que exige uma sessão válida – e nada mais.

O tipo de papel inclui `viewer`, que a própria UI rotula “Visualização” (`ROLE\_LABELS.viewer`). Mas um `viewer` consegue `POST /api/crm/pipelines` criar um funil, `POST /api/crm/tags` criar etiquetas e, via rota de item, `DELETE /api/crm/contacts/<id>` apagar qualquer contato e `DELETE /api/orders/<id>` apagar pedidos. O rótulo comercial diz o que o papel não cumpre, e nenhum gate de frontend esconde nada: `pageHead()` renderiza o botão “+ Novo usuário” incondicionalmente.

Busca por `requireRole` em todo `src/` retorna apenas 4 arquivos: users/index.ts, users/[id].ts, settings.ts e whatsapp/session.ts.

Evidência

```

```ts
src/server/routeFactory.ts:39-57
 export function createCollectionRoutes(table: string, shape: TableShape) {
 const GET: APIRoute = async () => {
 const items = await listEntities(getDb(), table, shape);
 return json(items);
 };
 const POST: APIRoute = async (context) => {
 /* sem requireRole */
 };

src/server/routeFactory.ts:59-80
 export function createItemRoutes(table: string, shape: TableShape) {
 const PUT: APIRoute = async (context) => { /* ... */ };
 const DELETE: APIRoute = async (context) => { /* ... */ };

src/domain/crm.ts:5
 export type Role = 'viewer' | 'agent' | 'manager' | 'admin';

src/ui/views/crm/CrmEquipeView.ts:12
 viewer: 'Visualização',

src/ui/views/crm/CrmEquipeView.ts:42-47 (botão sem gate)
 function pageHead(): string {
 const btn = '<button class="btn btn-primary" id="new-user">+ Novo usuário</button>';
 return section('Equipe', 'Usuários e papéis do sistema', btn);
 }
...

```

**## Impacto**

Qualquer conta de menor privilégio – inclusive uma criada para papel somente-visualização – tem capacidade de escrita e exclusão sobre toda a base de CRM e ERP: apagar contatos, conversas, mensagens, negócios, pedidos e modificar a configuração de funil/etiquetas/agenda. Qualquer sessão vazada ou conta descartada vira destruição em massa.

**## Sugestão de correção**

Dar a `createCollectionRoutes`/`createItemRoutes` uma lista de papéis permitidos por operação e chamar `requireRole` antes de tocar o DL. Sugestão: leitura = todos os papéis; escrita = `agent`, `manager`, `admin`; exclusão = `manager`, `admin`.

**## Critérios de aceite**

- [ ] POST /api/crm/pipelines com sessão `viewer` retorna 403 `papel\_insuficiente`.
  - [ ] DELETE /api/crm/contacts/:id com sessão `viewer` retorna 403 e o registro continua no banco.
  - [ ] POST /api/orders com sessão `viewer` retorna 403.
  - [ ] DELETE /api/crm/pipelines/:id com sessão `agent` retorna 403.
  - [ ] Uma entrada `role\_denied` é gravada em auth\_audit para cada negativa (o authz.ts já faz isso).
  - [ ] A UI de cada tela reflete a mesma matriz de permissões.
- FIM ISSUE 3 ---

**ISSUE 4**

--- ISSUE 4 ---

# [Segurança] IDOR: createItemRoutes faz PUT/DELETE por id sem checar posse nem papel

**\*\*Labels sugeridas:\*\*** `security` · `high` · `idor` · `broken-access-control`

**## Problema**

`createItemRoutes` resolve o registro pelo `context.params.id` e vai direto para `updateEntity`/`deleteEntity`. Não há comparação de `assignedUserId`/`assigneeUserId` nem gate de papel – a mesma lacuna que a issue anterior apontou, agora no eixo de posse.

Todos os arquivos `[id].ts` são aliases de duas linhas dessa fábrica, e foram percorridos um a um: contatos, conversas, mensagens, negócios, tarefas, pipelines, etapas, respostas-rápidas, agenda, catálogo, atividades, notas de conversa, etiquetas, tipos de agenda, clientes, pedidos, produtos, ingredientes, componentes e usuários. Efeito concreto: um usuário com papel `viewer` apaga qualquer contato, conversa, negócio ou pedido pelo id.

E como as listagens devolvem a base inteira (o projeto é single-tenant por desenho), os ids necessários para o ataque saem eles próprios das respostas de listagem – não é preciso adivinhar nada.

**## Evidência**

```

``ts
src/server/routeFactory.ts:60-72
 const PUT: APIRoute = async (context) => {
 const patch = (await context.request.json()) as { id?: string; } & Record<string, unknown>;
 const saved = await updateEntity(
 getDb(), table, shape, context.params.id!, patch
);
 return saved ? json(saved) : notFound();
 };

src/server/routeFactory.ts:74-77
 const DELETE: APIRoute = async (context) => {
 await deleteEntity(getDb(), table, context.params.id!);
 return json({ ok: true });
 };

```

Exemplo de alias – src/pages/api/crm/contacts/[id].ts:3  
 export const { PUT, DELETE } = createItemRoutes(CONTACTS\_TABLE, CONTACTS\_SHAPE);

Exemplo de alias – src/pages/api/orders/[id].ts:3  
 export const { PUT, DELETE } = createItemRoutes(ORDERS\_TABLE, ORDERS\_SHAPE);  
 ...

## ## Impacto

Qualquer sessão autenticada pode apagar ou alterar qualquer registro de qualquer entidade por id – incluindo conversas e mensagens do WhatsApp, negócios e pedidos. Combinado com a listagem sem filtro, é um read-write-destroy global para a base do CRM, sem necessidade de privilégio nenhum além de estar logado.

## ## Sugestão de correção

Antes do write, resolver o registro existente e exigir: (a) papel mínimo para a operação e (b) quando a tabela tiver coluna de dono, `registro.<dono> === user.id` \*\*ou\*\* papel `manager`/`admin`. Fazer isso na fábrica, para não depender de cada arquivo de rota.

## ## Critérios de aceite

- [ ] DELETE /api/crm/contacts/:id de um contato atribuído a outro usuário, por sessão `agent`, retorna 403 e não apaga o registro.
  - [ ] O mesmo DELETE por sessão `manager` ou `admin` funciona.
  - [ ] PUT /api/crm/tasks/:id de tarefa com outro `assigneeUserId`, por sessão `agent`, retorna 403.
  - [ ] As rotas sem coluna de dono (ex.: tags, produtos) exigem papel mínimo para escrita/exclusão.
  - [ ] A verificação está na fábrica (um único lugar), não replicada em 24 arquivos.
  - [ ] Teste automatizado cobrindo ao menos uma entidade com dono (contacts) e uma sem (tags).
- FIM ISSUE 4 ---

## ISSUE 5

--- ISSUE 5 ---

# [Segurança] IDOR no envio de WhatsApp: qualquer sessão envia mensagem em qualquer conversa

**Labels sugeridas:** `security` · `high` · `idor` · `whatsapp`

## ## Problema

`POST /api/whatsapp/send` recebe `conversationId` no corpo e resolve a conversa por id. `loadWahaConversation` valida apenas que a conversa existe e que `channel === 'whatsapp'` – **nunca** compara `conversation.assignedUserId` com o usuário chamador. A rota também não tem `requireRole`, só sessão.

Resultado: qualquer conta autenticada, de qualquer papel (inclusive `viewer`), pode listar as conversas – as listagens não filtram nada – e então disparar WhatsApp para qualquer cliente do negócio, em nome da empresa. O canal é externo e comercial: a mensagem chega ao celular do cliente final.

## ## Evidência

```

``ts
src/pages/api/whatsapp/send.ts:15-16
 const user = await userFromToken(db, context.request);
 if (!user) return json({ error: 'sessao_invalida' }, 401);

src/pages/api/whatsapp/send.ts:26-31
 const message = await sendWahaText(db, client, {
 conversationId: parsed.value.conversationId,
 text: parsed.value.text,
 userId: user.id,
 replyTo: parsed.value.replyTo
 });

```

```
src/server/wahaSend.ts:62-79 (sem checagem de dono)
 async function loadWahaConversation(
 db: Database, conversationId: string
): Promise<Conversation> {
 const conversation = await getEntity<Conversation>(
 db, CONVERSATIONS_TABLE, CONVERSATIONS_SHAPE, conversationId
);
 if (!conversation) {
 throw new WahaSendError('conversation_not_found', 'Conversa não encontrada');
 }
 if (conversation.channel !== 'whatsapp') {
 throw new WahaSendError('wrong_channel', 'Esta conversa não é do canal WhatsApp');
 }
 return conversation; // <-- assignedUserId nunca é comparado
 }

$ grep -c assignedUserId src/server/wahaSend.ts
0
...

```

### ## Impacto

Abuso de canal comercial a partir de conta de menor privilégio: envio de mensagens não autorizadas (golpe, phishing, oferta indevida) para a base de clientes, com custo por mensagem e sem rasto de autorização além do `createdBy` na linha enviada.

### ## Sugestão de correção

Exigir `conversation.assignedUserId === user.id || ['manager','admin'].includes(user.role)` em `loadWahaConversation`, e aplicar `requireRole(['agent','manager','admin'])` na rota.

### ## Critérios de aceite

- [ ] POST /api/whatsapp/send com `conversationId` de conversa atribuída a outro usuário, por sessão `agent`, retorna 403.
  - [ ] O mesmo envio por sessão `manager` ou `admin` funciona.
  - [ ] Uma sessão `viewer` recebe 403 na rota mesmo em conversa própria.
  - [ ] A mensagem de erro não distingue “não existe” de “não é sua” (evita enumeração de conversas).
  - [ ] Teste automatizado em tests/server cobrindo os dois caminhos.
- FIM ISSUE 5 ---

## ISSUE 6

--- ISSUE 6 ---

# [Segurança] Injeção de HTML armazenada via nome de ingrediente/contato em openModal e no Dashboard

**Labels sugeridas:** `security` · `high` · `xss` · `stored-xss`

### ## Problema

`openModal` interpola `options.title` diretamente no `innerHTML` do backdrop, sem escape. Dois callers passam dado vindo do banco sem escapar: o nome do contato (`Histórico de \${contact.name}`) e o nome do ingrediente (`Movimentar: \${ingredient.name}`).

Separadamente, `DashboardView.lowStockRow` interpola `\${i.name}` cru no innerHTML – e o Dashboard é a primeira tela que qualquer usuário vê ao entrar, o que dá alcance máximo ao payload.

Como **nenhuma** rota de ingrediente ou contato exige papel mínimo para escrita, qualquer sessão autenticada (inclusive `viewer`) planta o payload com um `PUT /api/ingredients/:id` e ele é renderizado depois para todos os usuários que abrirem aquele formulário ou o Dashboard. A tela de Equipe mostra o padrão correto logo ao lado (`CrmEquipeView` escapa `me.name`/`me.email` nas linhas 65-66), então trata-se de views que esqueceram o escape.

### ## Evidência

```
src/ui/Modal.ts:28-31
 backdrop.innerHTML = `
 <div class="modal" role="dialog" aria-modal="true">
 <div class="modal-head">
 <h3>${options.title}</h3>

src/ui/views/crm/CrmContatosView.ts:302
 openModal({ title: `Histórico de ${contact?.name ?? 'contato'}`, bodyHtml });

src/ui/views/StockView.ts:169-170
 const title = `Movimentar: ${ingredient.name}`;

```



```
const modal = openModal({ title, bodyHtml: formHtml(ingredient) });

src/ui/views/DashboardView.ts:86-88
function lowStockRow(i: Ingredient): string {
 return `<div class="calc-row">${i.name}
 ${i.stock} / ${i.minStock} ${i.unit}</div>`;
}
```

Contraste – o mesmo dado escapado em src/ui/views/crm/CrmEquipeView.ts:65-66

```
const name = escapeHtml(me.name);
... const email = escapeHtml(me.email);
...
```

## ## Impacto

Injeção de HTML armazenada, com leitura garantida: o Dashboard é a home de qualquer sessão. A CSP atual (`script-src 'self'`, sem `unsafe-inline`) bloqueia handlers inline, então o resultado hoje é injeção de HTML/atributo – phishing visual, injeção de formulário, sequestro de clique – e não execução de script. Se a CSP for relaxada, ou se o token em `localStorage` (ApiAuthRepository.ts:56) for lido por um gadget, o mesmo bug vira sequestro de conta completo.

## ## Sugestão de correção

Escarpar `options.title` dentro de Modal.ts – ponto único que cobre os dois callers – e escapar `i.name`/`i.unit` em DashboardView. Idealmente adicionar uma regra de lint que proíba interpolação em `innerHTML` sem `escapeHtml`/`escapeAttr`.

## ## Critérios de aceite

- [ ] PUT /api/ingredients/:id com name='<img src=x onerror=alert(1)>' e a abertura do formulário de movimentação mostra o texto literal, sem elemento criado.
- [ ] O mesmo nome malicioso no Dashboard aparece escapado.
- [ ] Modal.ts escapa o title (teste de unidade com title contendo '<b>' e '').
- [ ] Existe regra de lint/CI que falha se uma atribuição a innerHTML interpolar variável sem um dos helpers de escape.
- [ ] Teste de UI (happy-dom) cobrindo Modal.ts com title contendo HTML.
- FIM ISSUE 6 ---

## ISSUE 7

--- ISSUE 7 ---

# [Segurança] escapeHtml não escapa aspas e é usado em contexto de atributo (quebra de atributo)

**Labels sugeridas:** `security` · `medium` · `xss` · `escaping`

## ## Problema

`escapeHtml` (src/domain/format.ts:33-37) escapa via `textContent` → `innerHTML`, o que trata `&`, `<` e `>` mas **não** nem `` – aspas só importam em contexto de atributo, e o helper não pode saber onde o resultado será usado. É por isso que o projeto tem `escapeAttr` (format.ts:39) e `escapeAttr` (dom.ts:65).

Em três pontos o valor vai para dentro de aspas de atributo usando o helper errado:

- `CrmEtiquetasView:91` – `class="chip chip-\${escapeHtml(tag.color)}"`, e `tag.color` é gravável por qualquer sessão autenticada via `PUT /api/crm/tags/:id` (nenhum gate de papel).
- `ProductsView:508-510` – `value="\${escapeHtml(value)}"`, enquanto o `crmUi.textField` equivalente usa `escapeAttr` (crmUi.ts:57).
- `CrmWhatsAppView:308-310` – `src="\${escapeHtml(qr)}"`, onde `qr` vem do engine WAHA (alcançável por HTTP simples em dev).

Um `` no payload fecha o atributo e abre Markup. Some-se a isso que existem **dois** helpers de escape de atributo divergentes no projeto, o que é a causa raiz do padrão inconsistente.

A mesma classe de bug aparece nos `numberField` (crmUi.ts:69, ProductsView:521, IngredientsView:195, ComponentsView:316, SettingsView:97), que interpolam `value` cru, e no `rowButton` (crmUi.ts:129), que interpola o atributo `data-\*` sem escape. Os valores atuais são numéricos ou ids, mas o servidor não valida tipo – `entityToRow` só restringe nomes de coluna.

## ## Evidência

```
```ts
src/domain/format.ts:33-37
export function escapeHtml(text: string): string {
  const div = document.createElement('div');
  div.textContent = text ?? '';
  return div.innerHTML;
}
// -> escapa & < > ; NÃO escapa " nem `
```

```

src/domain/format.ts:39-42 (o helper correto, que existe)
  export function escapeAtrib(text: string): string {
    return escapeHtml(text)
      .replace(/"/g, "&quot;")
      .replace(/\'/g, "&#x27;")
  }

src/ui/dom.ts:65-67 (o segundo helper, divergente)
  export function escapeAttr(text: string): string {
    return String(text).replace(/"/g, '&quot;');
  }

src/ui/views/crm/CrmEtiquetasView.ts:91
  `<span class="chip chip-${escapeHtml(tag.color)}">` +

src/ui/views/ProductsView.ts:508-510
  <input class="input" name="${name}" value="${escapeHtml(value)}" required>

src/ui/views/crm/CrmWhatsAppView.ts:308-310
  const safe = escapeHtml(qr);
  const tag = qr.startsWith('data:') ? ``

src/ui/views/crm/crmUi.ts:69
  ... const attrsB = ` name="${name}" value="${value}" required`;

```

Impacto

Injeção de atributo/HTML a partir de `tag.color` (controlável por qualquer sessão) e dos valores de produto gravados via API. Sob a CSP atual o efeito para em HTML, mas a superfície fica disponível caso a CSP mude.

Sugestão de correção

Trocar por `escapeAtrib`/`escapeAttr` nos pontos de atributo e unificar os dois helpers em um único `escapeAttr`, removendo a duplicação. Escapar também `value` nos `numberField` e o atributo `data-\*` do `rowButton`.

Critérios de aceite

- [] PUT /api/crm/tags/:id com color='x' onmouseover='alert(1)' renderiza o atributo class intacto, sem atributo extra.
 - [] ProductsView usa o helper de atributo no value do input.
 - [] Existe exatamente um helper de escape de atributo em src/ (grep por 'export function escape' retorna três funções: html, attr e nada mais).
 - [] numberField (crmUi.ts:69, ProductsView:521, IngredientsView:195, ComponentsView:316, SettingsView:97) escapa o value.
 - [] rowButton (crmUi.ts:129) escapa o valor do atributo data-*.
 - [] Teste de unidade que falhe se escapeHtml aplicado em atributo receber aspas sem escapar.
- FIM ISSUE 7 ---

ISSUE 8

--- ISSUE 8 ---

[Segurança] Credencial de administrador padrão (`admin123`) documentada no repositório, com salt fixo no código

****Labels sugeridas:**** `security` · `high` · `secrets` · `default-credentials`

Problema

A migration de seed grava um usuário administrador com a senha `admin123`, usando o salt FIXO `deskcomm-seed-v1` – e esse mesmo salt está no código-fonte, como valor default do parâmetro `salt` de `verifyPassword`. O hash também está versionado (a migration 0019 o usa de propósito, para ser idempotente).

O resultado é um credencial de administrador ****reproduzível por qualquer pessoa com o repositório****: e-mail `admin@deskcomm.local`, senha `admin123`, sal conhecido. Além disso, `verifyPassword` cai de volta nesse sal padrão sempre que a linha não tem `passwordSalt`, o que amplia o alcance do valor fixo.

Há uma mitigação real e bem feita – `mustChangePassword=1` (migrations 0020/0021) e o middleware respondendo 403 `troca\_de\_senha\_obrigatoria` fora de uma allowlist de três rotas. O que não existe é validação de startup que rejeite o credencial de seed: o `\*\*login` retorna 200 e um token de sessão válido**; só a API fica fechada depois disso.

Evidência

```

`ts
migrations/0004_crm_seed.sql:5-9
  -- Admin user – senha padrão "admin123" (PBKDF2-SHA256, 100k iterações, sal
  -- "deskcomm-seed-v1"). Troque na primeira sessão pela tela de Equipe.
  INSERT OR IGNORE INTO users (id, name, email, passwordHash, role, createdAt) VALUES

```

```
( 'seed-user-admin', 'Administrador', 'admin@deskcomm.local',
  '022d504d3b3433f2cde7ac9185a4e1d340e67ed70a943dbc4ef14bf8c3174a00', 'admin', '2026-01-01T00:00:00.000Z' );

src/server/auth.ts:26
  const SALT = 'deskcomm-seed-v1';

src/server/auth.ts:48-52
  export async function verifyPassword(
    password: string,
    storedHash: string,
    salt: string = SALT      // <-- default é o sal publicado
  ): Promise<boolean> {

A mitigação que existe – src/middleware.ts:138-144
  async function blockedByPasswordChange(user: User, pathname: string): Promise<boolean> {
    if (PASSWORD_CHANGE_ALLOWED.has(pathname)) return false;
    return Number(user.mustChangePassword ?? 0) === 1;
  }
  ...
```

Impacto

Em qualquer ambiente novo onde `npm run db:seed:remote` roda sem o operador trocar a senha, existe um login de administrador reproduzível a partir do repositório. O `mustChangePassword` reduz o alcance (a API fica 403), mas o atacante ainda obtém uma sessão autenticada válida – suficiente para as três rotas da allowlist (`/api/auth/me`, `/api/auth/logout`, `/api/auth/change-password`) e para enumerar o estado da instalação.

Sugestão de correção

Deixar de semear senha. Gerar o admin no primeiro boot com senha aleatória exibida uma única vez, ou exigir `ADMIN\_INITIAL\_PASSWORD` no env. Adicionar asserção de startup que aborta se o `passwordHash` do seed ainda estiver presente, e remover o sal fixo do default de `verifyPassword`.

Critérios de aceite

- [] Nenhuma migration versionada contém senha de administrador em texto plano.
 - [] Um banco novo semeado não permite login com 'admin123'.
 - [] verifyPassword não tem mais o sal fixo como valor default (ou o default é explicitamente rejeitado).
 - [] Existe asserção de startup que falha se users.passwordHash do seed-user-admin estiver presente.
 - [] O README descreve o fluxo de criação do primeiro admin sem credencial padrão.
- FIM ISSUE 8 ---

ISSUE 9

--- ISSUE 9 ---

[Segurança] Defaults de segredo e app vendorizado: placeholder de HMAC aceito como segredo e DeskcommCRM inteiro no repositório

****Labels sugeridas:**** `security` · `medium` · `secrets` · `supply-chain` · `housekeeping`

Problema

Duas coisas de higiene de segredo, ambas verificadas:

**** (a) Placeholder aceito como segredo. **** `readWahaWebhookConfig` usa um helper `text()` que só checa "string não vazia", então o valor de instrução `gere-um-segredo-por-ambiente-openssl-rand-hex-32` presente no `.dev.vars` passa a valer como segredo HMAC válido. Neste ambiente o efeito é fail-closed (bom, porque `WAHA\_WEBHOOK\_REQUIRE\_SIGNATURE="true"` está ligado), mas o critério de validade é frouxo: "não vazio" é o único teste. O contraste é o próprio projeto: `readWahaConfig` já rejeita explicitamente `WAHA\_DEV\_PLACEHOLDER\_KEY` – o padrão certo existe, só não foi aplicado aqui. E o `.dev.vars.example` deixa o campo ****vazio****, o que desliga a verificação inteira.

Além disso, o `.dev.vars` da árvore de trabalho contém uma `WAHA\_API\_KEY` real de 64 hex (não versionada – confirmado com `git ls-files --error-unmatch` e `git log --all -S`).

**** (b) Segundo aplicativo inteiro no repositório. **** `DeskcommCRM-RecipeCosting/` é uma cópia completa de outro produto (Next.js + Supabase + Sentry) com 4807 arquivos rastreados, incluindo `.env.example` (25 KB), `.env.hostgator.example` (17 KB), `supabase/migrations/`, `Dockerfile\*` e `docker-compose.prod.yml`. Os JWTs que aparecem ali são fixtures públicos do Supabase e placeholders (`chave-de-mentira`) – nenhum segredo real foi encontrado. O problema é estrutural: superfície de segredo e de ataque sem revisão, e ruído para qualquer scanner/DAST.

Evidência

```
``ts
.dev.vars:31
  WAHA_HMAC_SECRET="gere-um-segredo-por-ambiente-openssl-rand-hex-32"
```

```
src/server/wahaWebhook.ts:29-35
  export function readWahaWebhookConfig(source: unknown): WahaWebhookConfig {
    const record = source as Record<string, unknown> | null | undefined;
    const secret = text(record?.WAHA_HMAC_SECRET); // <-- só exige não-vazio
    const flag = record?.WAHA_WEBHOOK_REQUIRE_SIGNATURE ?? '';
    const requireSignature = String(flag) === 'true';
    return { hmacSecret: secret, requireSignature };
  }

O padrão correto que já existe no projeto – src/server/waha.ts:60-70
  export function readWahaConfig(source: unknown): WahaConfig | null {
    const url = text(record?.WAHA_API_BASE_URL);
    const apiKey = text(record?.WAHA_API_KEY);
    if (!url || !apiKey || apiKey === WAHA_DEV_PLACEHOLDER_KEY) return null;
  }

.dev.vars:9
  WAHA_API_KEY="7ff62014c4d63e715d9efeffc400964dff94299ed806165134b2744dff2ec818"

DeskcommCRM-RecipeCosting/.env.example (25.097 bytes, versionado)
DeskcommCRM-RecipeCosting/.env.hostgator.example (17.002 bytes, versionado)
DeskcommCRM-RecipeCosting/docker-compose.prod.yml (produção de outro app, versionado)
git ls-files DeskcommCRM-RecipeCosting | wc -l -> 4807
...

## Impacto

Nenhum segredo real exposto foi encontrado – este achado é sobre fraza de controle e higiene, não sobre um breach. O risco concreto é (a): uma configuração com segredo placeholder-but-non-empty não é rejeitada, e o `.dev.vars.example` ensaia justamente a configuração que DESLIGA a verificação de assinatura. O risco de (b) é desupply chain: qualquer segredo que entre nessa árvore no futuro é commitado junto com o app principal, sem revisão.

## Sugestão de correção

(a) Rejeitar valores conhecidos de placeholder e segredos com menos de 32 bytes em `readWahaWebhookConfig`, e preencher o exemplo com um valor que force o operador a gerar o seu. (b) Remover `DeskcommCRM-RecipeCosting/` do repositório (ou movê-lo para repo próprio / submódulo explícito) e adicionar gitleaks/trufflehog no CI.

## Critérios de aceite

- [ ] readWahaWebhookConfig devolve null (ou o chamador recusa) quando WAHA_HMAC_SECRET é o placeholder ou tem menos de 32 bytes.
- [ ] .dev.vars.example traz um valor que não habilita a assinatura por acidente.
- [ ] git ls-files DeskcommCRM-RecipeCosting retorna vazio, e o caminho está no .gitignore.
- [ ] O CI roda um scanner de segredo (gitleaks/trufflehog) e falha em alta.
- [ ] Existe um teste unitário para readWahaWebhookConfig cobrindo placeholder, vazio e tamanho curto.
--- FIM ISSUE 9 ---
```

ISSUE 10

```
--- ISSUE 10 ---
# [Segurança] Sessão em localStorage e token em query string no SSE amplificam qualquer injeção de HTML

**Labels sugeridas:** `security` · `medium` · `session` · `defense-in-depth`

## Problema

O token Bearer vive em `localStorage` (`ApiAuthRepository.ts:20,56`), legível por qualquer script que rode na origem. E, porque o `EventSource` não consegue mandar header `Authorization`, o middleware abre uma exceção e aceita o token por query string em `/api/crm/events` (`middleware.ts:75-87`, `crm/events.ts:137`), com `Access-Control-Allow-Origin: *` na resposta.

Hoje a CSP (`script-src 'self'`, sem `unsafe-inline`) impede que os sinks de innerHTML verificados executem script, então o pior caso é injeção de HTML. O ponto do achado é o nível de dano: se a CSP relaxar, ou se um sink passar a aceitar `javascript:`, o mesmo bug vira sequestro de conta completo em vez de deformação de HTML. E o token em URL vaza para log de acesso, header `Referer` e histórico do navegador – risco que o comentário do próprio middleware reconhece ao justificar a exceção.

## Evidência

```ts
src/repositories/ApiAuthRepository.ts:20
 return localStorage.getItem(this.tokenKey);

src/repositories/ApiAuthRepository.ts:56
 localStorage.setItem(this.tokenKey, token);
```

```
src/middleware.ts:70-75 (o comentário admite o trade-off)
// The SSE endpoint is the one route a browser can only reach with the token in
// the query string: an EventSource cannot set an Authorization header, so the
// client puts `?token=` in the URL (see sseEventUrl in the CRM views). Every
// other route stays Bearer-only – a token in a URL lands in access logs,
// Referer headers and browser history.
```

```
src/pages/api/crm/events.ts:137
const queryToken = url.searchParams.get('token');
```

```
src/pages/api/crm/events.ts:172
... 'Access-Control-Allow-Origin': '*'
```

## Impacto

Risco de defence-in-depth: não é explorável isoladamente, mas é o que transformaria as injeções de HTML das outras issues em comprometimento total de sessão. O token em query string, isoladamente, é exposição de credencial em log/Referer/histórico – em especial porque o SSE também faz broadcast de todas as mensagens e telefones do sistema.

## Sugestão de correção

Migrar a sessão para cookie `HttpOnly; Secure; SameSite=Strict`, o que resolve de uma vez o token-em-URL do SSE, o `Access-Control-Allow-Origin: \*` e a leitura do token por script.

## Critérios de aceite

- [ ] Nenhuma rota aceita token por query string (grep por searchParams.get('token') não retorna nada em src/).
- [ ] A sessão é um cookie HttpOnly: o teste frontend não consegue ler o token via document.cookie.
- [ ] /api/crm/events não envia Access-Control-Allow-Origin.
- [ ] O middleware não tem mais a exceção SSE\_PATH.
- [ ] Logout invalida o cookie (e a linha em sessions) e a sessão não sobrevive ao cookie-clear.

--- FIM ISSUE 10 ---

## ISSUE 11

--- ISSUE 11 ---

# [Segurança] GET /api/settings escreve no banco e faz merge cego de colunas arbitrárias

**\*\*Labels sugeridas:\*\*** `security` · `medium` · `broken-access-control` · `hardening`

## Problema

A rota de leitura de configurações também **\*\*escreve\*\***: quando o registro `global` ainda não existe, o GET faz `INSERT OR REPLACE` com `DEFAULT\_SETTINGS`. Estado que muda em resposta a um GET, sem gate de papel nenhum (o GET não chama `requireRole`).

O PUT tem gate de `[admin, 'manager']`, mas faz `{ ...current, ...patch }` sem validar o patch contra um schema, e `settings` é a **\*\*única\*\*** tabela sem lista `columns` no shape – `keepAllowed` não filtra nada (o próprio comentário em mapping.ts registra isso). O resultado é que quem pode escrever escolhe quais colunas da linha `settings` sobrescreve, inclusive nomes que a coluna não tem.

## Evidência

```
...ts
src/pages/api/settings.ts:13-19
export const GET: APIRoute = async () => {
 const db = getDb();
 const existing = await readSettings(db);
 if (existing) return json(existing);
 await writeSettings(db, DEFAULT_SETTINGS); // <-- GET que escreve
 return json(DEFAULT_SETTINGS);
};

src/pages/api/settings.ts:25-28
const patch: Partial<Settings> = await context.request.json();
const current = (await readSettings(db)) ?? DEFAULT_SETTINGS;
const merged = { ...current, ...patch }; // <-- merge sem validação
await writeSettings(db, merged);

src/server/mapping.ts:9-10 (a própria ressalva)
// Omitting `columns` stays permissive; only `settings` does that, because
// its route has no shape of its own.

src/server/mapping.ts:48-53
function keepAllowed(
 entity: Record<string, unknown>,
```

```

 columns: string[] | undefined
): Record<string, unknown> {
 if (!columns) return { ...entity };
 },
 ...

```

### ## Impacto

Um `manager` (papel não-administrador) consegue gravar colunas arbitrárias na configuração global, com efeito direto em cálculo de custos, preços e margens exibidos no Ateliê. O GET-que-escreve também torna a rota não-idempotente e cria estado em um read, o que complica cache e auditoria.

### ## Sugestão de correção

Mover a criação do registro default para uma migration/seed, deixar o GET puramente de leitura, dar a `settings` uma lista explícita de colunas no shape, e validar o patch contra um schema antes do merge.

### ## Critérios de aceite

- [ ] GET /api/settings não executa nenhuma instrução de escrita (verificável com um D1 fake que conte as calls).
  - [ ] settings tem shape.columns explícito; uma chave fora da lista é descartada no merge.
  - [ ] PUT com chave desconhecida no corpo não altera a linha persistida.
  - [ ] Existe schema de validação (zod ou equivalente) para o patch de settings.
- FIM ISSUE 11 ---

## ISSUE 12

--- ISSUE 12 ---

# [Segurança] Gate de papel do frontend divergente do endpoint (e ROLE\_RANK morto)

**Labels sugeridas:** `security` · `medium` · `authz` · `consistency`

### ## Problema

O gate de papel não é uma fonte única, e isso já produziu erro nos dois sentidos.

**(a)** UI mais permissiva que o servidor. **CrmequipeView.actionButtons** libera o botão Excluir para `manager`, mas `DELETE /api/users/[id]` exige `['admin']`. O servidor está certo e a UI mente – o operador clica e leva 403.

**(b)** UI sem gate nenhum. `pageHead()` renderiza “+ Novo usuário” incondicionalmente, sem olhar o papel: um `viewer` vê e clica.

**(c)** Hierarquia de papéis declarada e nunca usada. `ROLE_RANK` (`viewer:0 ... admin:3`) está definido em `crm.ts` e não tem nenhum consumidor em `src/` ou `tests/`. Os gates comparam listas explícitas, então o modelo de papéis é informal: o que vale é apenas o que cada lista escrita à mão disser – e é exatamente aí que a escalada de privilégio da issue #1 passou.

### ## Evidência

```

`ts
src/ui/views/crm/CrmEquipeView.ts:102-104
 const canRemove =
 (me?.role === 'admin' || me?.role === 'manager') &&
 (me?.id !== user.id || others.length === 0);

src/pages/api/users/[id].ts:50 (servidor mais estrito)
 const denied = await requireRole(context, ADMINS);

src/ui/views/crm/CrmEquipeView.ts:42-47 (sem gate nenhum)
 function pageHead(): string {
 const btn = '<button class="btn btn-primary" id="new-user">+ Novo usuário</button>';
 return section('Equipe', 'Usuários e papéis do sistema', btn);
 }

src/domain/crm.ts:166-171 (hierarquia morta)
 export const ROLE_RANK: Record<Role, number> = {
 viewer: 0,
 agent: 1,
 manager: 2,
 admin: 3
 };
 // grep em src/ e tests/: apenas a definição, nenhum consumidor.
...

```

### ## Impacto

Não há bypass por si só (o servidor sempre decide), mas a divergência confunde o operador sobre o que é permitido, e a duplicação da matriz de papéis em literais é a causa estrutural da escalada de privilégio – o mesmo arquivo

```
`users/index.ts` valida papel corretamente no PUT e incorretamente no POST.
```

#### ## Sugestão de correção

Extrair uma matriz `can(user, action)` em módulo único, consumida pela UI (para desabilitar/esconder) e pelo servidor (via `requireRole`). Usar `ROLE\_RANK[user.role] >= ROLE\_RANK[required]` em `requireRole`, ou remover a hierarquia e a menção a ranking.

#### ## Critérios de aceite

- [ ] Existe uma função/matriz única descrevendo (papel, ação) e ela é usada por authz.ts e pelas views.
  - [ ] O botão Excluir só aparece quando o endpoint aceita o papel (manager não vê para usuário).
  - [ ] O botão '+ Novo usuário' é condicionado ao papel na renderização.
  - [ ] ROLE\_RANK tem um consumidor, ou deixou de existir.
  - [ ] Teste garante que UI e servidor concordam para (viewer, agent, manager, admin).
- FIM ISSUE 12 ---